

Weather Application using API Integration

Submitted By

Ardra Rajesh

Internship

CODEVEDX Python Programming Internship

Department

B.Tech Computer Science & Engineering

Date

02 July 2026

Table of Contents

1. Introduction
2. Objectives
3. Features
4. Technologies Used
5. Python Modules Used
6. Project Workflow
7. Project Structure
8. Screenshots
9. Sample Output
10. Error Handling
11. Future Enhancements
12. Conclusion

Introduction

Weather information plays an important role in everyday life by helping people plan travel, outdoor activities, and daily routines. With the advancement of web services and APIs, weather data can now be accessed instantly from online sources and displayed in user-friendly applications.

The **Weather Application using API Integration** is a Python-based console application developed to retrieve and display real-time weather information for any city using the **OpenWeatherMap API**. The application provides useful weather details such as temperature, humidity, wind speed, weather conditions, and the date and time of retrieval.

Unlike a basic weather application, this project also includes an **Offline Mock Mode** containing weather data for more than **60 Indian cities** and several international locations. This allows the application to function without an internet connection or API key, making it suitable for demonstrations and testing.

The application follows a modular architecture by separating API communication, data processing, display, and controller logic. It also maintains search history using JSON files, records application logs, supports Celsius and Fahrenheit units, and implements comprehensive exception handling for reliable performance.

This project helped me gain practical experience in API integration, JSON processing, logging, modular programming, file handling, and exception handling using Python.

Objectives

The main objectives of this project are:

- To develop a weather application using Python.
- To integrate a real-time weather API.
- To display current weather information for any city.
- To provide offline weather data using Mock Mode.
- To maintain search history using JSON files.
- To implement logging for monitoring application activities.
- To improve programming skills using APIs, JSON, and exception handling.
- To understand real-world API integration using Python.

Features

Real-Time Weather Information

- Retrieve current weather data using the OpenWeatherMap API.
- Display city and country.
- Display temperature.
- Display feels-like temperature.
- Display humidity.
- Display wind speed.
- Display weather condition.
- Display retrieval date and time.

Offline Mock Mode

- Built-in weather data for more than 60 Indian cities.
- Includes popular international cities.
- Works without internet access.
- No API key required during testing.

Temperature Unit Switching

- Switch between Celsius and Fahrenheit.
- Automatically update displayed values.

Search History

- Store previous searches in a JSON file.
- Display search history.
- Maintain persistent records between sessions.

Logging

- Record application events.
- Save logs with timestamps.
- Maintain a weather_app.log file.
- Assist in debugging.

Modular Application Design

- API Layer
- Parsing Layer
- Display Layer
- Controller Layer

Additional Features

- User-friendly menu-driven interface.
- Live Mode and Mock Mode switching.
- JSON-based storage.
- Exception handling.
- Input validation.

Technologies Used

Technology	Purpose
Python 3	Programming Language
OpenWeatherMap API	Weather Data Provider
JSON	Search History Storage
VS Code	Code Editor
Git	Version Control
GitHub	Project Repository

Python Modules Used

urllib.request

Used to send HTTP requests to the weather API.

urllib.error

Used to handle API and network-related errors.

urllib.parse

Used to encode API request parameters.

json

Used for parsing API responses and storing search history.

logging

Used to record application events with timestamps.

datetime

Used to record the current date and time of weather retrieval.

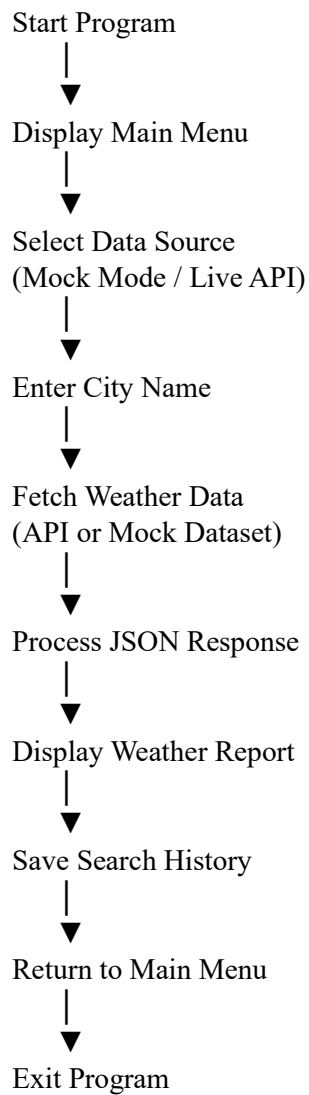
os

Used to check the existence of history files.

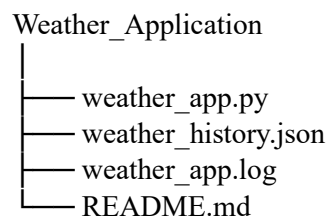
sys

Used for clean program termination and keyboard interrupt handling.

Project Workflow



Project Structure



Screenshots

```
PS C:\Users\rajes\Desktop\python\Weather> python weather_app.py

WEATHER APPLICATION
(mode: MOCK | units: metric)

=====
1. Get weather for a city
2. View search history
3. Toggle temperature units (C/F)
4. Toggle mock/live data source
5. Exit
=====

Enter your choice (1-5): 1
Enter city name: Bengaluru
2026-07-07 18:42:15 | INFO      | Looking up weather for 'Bengaluru'

=====
WEATHER REPORT: Bengaluru, IN
=====
Condition      : Clouds (Few Clouds)
Temperature    : 24.6°C
Feels Like     : 25.0°C
Humidity       : 60%
Wind Speed     : 3.8 m/s
Retrieved At   : 2026-07-07 18:42:15
=====

2026-07-07 18:42:15 | INFO      | Successfully displayed weather for 'Bengaluru'

=====
WEATHER APPLICATION
(mode: MOCK | units: metric)

=====
1. Get weather for a city
2. View search history
3. Toggle temperature units (C/F)
4. Toggle mock/live data source
5. Exit
=====

Enter your choice (1-5): 2

=====
SEARCH HISTORY
=====
1. Kochi, IN - 29.8° Rain (at 2026-07-05 20:28:04)
2. Thrissur, IN - 29.6° Clouds (at 2026-07-05 20:28:45)
3. Bengaluru, IN - 24.6° Clouds (at 2026-07-07 18:42:15)
=====

=====
WEATHER APPLICATION
(mode: MOCK | units: metric)

=====
1. Get weather for a city
2. View search history
3. Toggle temperature units (C/F)
4. Toggle mock/live data source
5. Exit
=====

Enter your choice (1-5): 3
2026-07-07 18:42:28 | INFO      | Units switched to 'imperial'

Units changed to: IMPERIAL

=====
WEATHER APPLICATION
(mode: MOCK | units: imperial)

=====
1. Get weather for a city
2. View search history
3. Toggle temperature units (C/F)
4. Toggle mock/live data source
5. Exit
=====

Enter your choice (1-5): 4
2026-07-07 18:42:33 | INFO      | Data source switched to LIVE (real API) mode

Data source is now: LIVE (real API)

Reminder: set a valid CONFIG['API_KEY'] for live mode to work.

=====
WEATHER APPLICATION
(mode: LIVE | units: imperial)

=====
1. Get weather for a city
2. View search history
3. Toggle temperature units (C/F)
4. Toggle mock/live data source
5. Exit
=====

Enter your choice (1-5): 5
2026-07-07 18:42:45 | INFO      | Weather Application exited by user

Goodbye!
```

Sample Output

WEATHER APPLICATION

Mode : Mock

Units: Celsius

1. Get Weather for a City
2. View Search History
3. Change Temperature Unit
4. Switch Data Source
5. Exit

Enter your choice: 1

Enter city name: London

Weather Report

City : London

Country : GB

Condition : Light Rain

Temperature : 17.2 °C

Feels Like : 17.0 °C

Humidity : 65%

Wind Speed : 4.1 m/s

Retrieved At : 2026-07-05 14:38:14

Error Handling

To make the application reliable and user-friendly, several validation and exception handling techniques have been implemented.

The system checks for:

- Empty city names.
- Invalid API keys.
- City not found.
- Internet connection failures.
- Network timeout errors.
- Invalid API responses.
- Corrupted JSON history files.
- Missing history files (created automatically).

- Invalid menu selections.
- Keyboard interruption during execution.

These validations ensure smooth execution and improve the overall reliability of the application.

Future Enhancements

The project can be further improved by adding:

- 5-day and 7-day weather forecasts.
- GPS-based weather search.
- Multiple weather service providers.
- Export search history to CSV or Excel.
- Weather charts and graphical reports.
- Graphical User Interface (GUI) using Tkinter.
- Web version using Flask or Django.
- Mobile application support.
- Voice-based weather search.

Conclusion

The **Weather Application using API Integration** was developed to gain practical experience in Python programming by implementing a real-world weather information system. Through this project, I learned how to integrate external APIs, process JSON responses, manage search history using JSON files, implement logging, and handle exceptions effectively.

Working on this project also improved my understanding of modular programming, HTTP requests, data processing, and user interaction. Features such as offline Mock Mode, real-time weather retrieval, search history management, temperature unit switching, and activity logging made the application practical and enhanced my problem-solving skills.

Overall, this project strengthened my confidence in building real-world Python applications that interact with web services and provided valuable hands-on experience in API integration and software development.